Git & GitHub — Complete Beginner's Cheat Sheet



A practical reference for everyday Git commands, branches, commits, and GitHub workflows.

Table of Contents

1. Basic Terminal Commands
2. Git Configuration
3. Creating a Repository
4. Working with Files and Commits
5. Remote Repositories
6. Branches
7. Merging Branches
8. Undoing Changes and Recovering Commits
9. Stashing Changes
10. GitHub Pull Request Process

---

1. Basic Terminal Commands

These commands help you navigate and manage files in Git Bash.

Command	Description
`ls`	Lists files and folders in the current directory.
`pwd`	Prints the path of the current directory.
`mkdir folder-name`	Creates a new directory.
`cd folder-name`	Changes the current directory to the specified folder.

Example:

```
mkdir git-practice
cd git-practice
pwd
ls
```

Note: `folder-name` is a placeholder. Replace it with your actual folder name.

---

2. Git Configuration

Git configuration defines your identity and preferences.

Command	Description
`git --version`	Displays the installed Git version.
`git config --global --list`	Lists your global Git configuration settings.
`git config --global user.name`	Displays your configured username.
`git config --global user.email`	Displays your configured email address.
`git config --global user.name "username"`	Sets your Git username globally.
`git config --global user.email "email"`	Sets your Git email globally.

Understanding the command

- `config`: Reads or changes Git settings.

- `--global` : Applies the setting to your Windows user account across repositories, unless overridden by a more specific configuration.
- `user.name` : The name recorded in your commits.
- `user.email` : The email recorded in your commits.

Example:

```
git config --global user.name "Borna Etminan"
git config --global user.email "you@example.com"
```

Replace the example email with your own email address.

---

3. Creating a Repository

A **repository (repo)** is a project that Git tracks, including its files and commit history.

Command	Description
`git init -b main`	Initializes a Git repository and names its initial branch `main`.

Understanding the command

- `init` : Initializes a new Git repository in the current directory.
- `-b` : Specifies the name of the initial branch.
- `main` : The branch name.

Example:

```
mkdir git-practice
cd git-practice
git init -b main
```

This creates a local Git repository. It does not automatically create a GitHub repository or make a commit.

---

4. Working with Files and Commits

A **commit** is a saved snapshot of your project at a particular point in its history.

File commands

Command	Description
`echo "text"`	Prints text in the terminal.
`echo "text" > file-name`	Writes text to a file, replacing its existing contents if it exists.
`echo "text" >> file-name`	Appends text to the end of a file.
`git status`	Shows the current branch and the status of changed, staged, and untracked files.
`git add file-name`	Stages changes to a specific file.
`git commit -m "message"`	Creates a commit using the staged changes.
`git diff`	Shows unstaged changes to tracked files.
`git diff --staged`	Shows changes staged for the next commit.

Understanding the command parts

- `echo`: Prints text.
- `>`: Writes to a file and replaces existing contents.
- `>>`: Appends to a file without replacing existing contents.
- `add`: Stages changes for a future commit.
- `commit`: Records a snapshot of staged changes.
- `-m`: Supplies the commit message directly in the command.
- `diff`: Displays differences between versions.
- `--staged`: Restricts the diff to changes in the staging area.

Example workflow:

```
echo "Hello Git" > README.md
git status
git add README.md
git diff --staged
git commit -m "Add README"
git status
```

The three main stages

1. **Working Directory:** Where you edit files.
2. **Staging Area:** Where you select changes for the next commit.
3. **Repository:** Where Git records commits.

Typical workflow:

`Edit files → git add → git commit`

Important: `git add` does not create a commit. It prepares changes to be committed.

---

5. Remote Repositories

A **remote repository** is a version of your repository hosted elsewhere, such as on GitHub. It allows you to share code and synchronize changes between your local computer and a remote server.

The name `origin` is a conventional name for the default remote; it is not a special Git keyword.

Command	Description
`git remote add origin URL`	Adds a remote repository named `origin`.
`git remote -v`	Displays configured remote URLs for fetching and pushing.
`git push -u origin main`	Pushes the local `main` branch to `origin` and sets its upstream branch.
`git clone URL`	Downloads a repository and creates a local working copy.

Understanding the command parts

- `remote`: Manages connections to remote repositories.
- `add`: Adds a new remote.
- `origin`: The remote's name.
- `-v`: Stands for verbose and displays additional information.
- `push`: Uploads local commits to a remote repository.
- `-u`: Sets the upstream relationship for future push and pull operations.
- `clone`: Copies an existing repository to your computer.

- `URL` : The address of the remote repository.

Example:

```
git remote add origin https://github.com/USERNAME/git-practice.git
git remote -v
git push -u origin main
```

Replace `USERNAME` with your GitHub username and use your actual repository URL.

Local vs. remote

- **Local repository:** Stored on your computer.
- **Remote repository:** Hosted on a service such as GitHub.
- `git push` : Sends local commits to the remote.
- `git pull` : Fetches remote changes and integrates them into the current branch.
- `git fetch` : Downloads remote updates without integrating them into your current branch.

---

6. Branches

A **branch** is an independent line of development. Branches let you work on features or fixes without immediately changing the main development line.

The `main` branch commonly represents the primary development line. Feature branches are often created for individual tasks.

Command	Description
`git branch --show-current`	Displays the current branch name.
`git switch -c branch-name`	Creates a new branch and switches to it.
`git switch branch-name`	Switches to an existing branch.

Understanding the command parts

- `branch` : Refers to branches or manages them.
- `--show-current` : Displays only the current branch name.
- `switch` : Changes the current branch.
- `-c` : Creates a new branch before switching to it.

Example:

```
git switch -c feature-login
```

Make your changes and commit them on `feature-login`. When you want to return to the main branch:

```
git switch main
```

Important: A commit made on a feature branch does not automatically become part of `main`. You must merge the branch or integrate its changes through another workflow.

---

7. Merging Branches

Merging combines changes from one branch into another. You normally switch to the branch that should receive the changes, then merge the source branch into it.

Command	Description
`git merge branch-name`	Merges the specified branch into the current branch.
`git log --oneline --graph --all`	Displays a compact, graphical history of commits across all local and known remote-tracking branches.

Understanding the command parts

- `merge`: Integrates changes from another branch.
- `log`: Displays commit history.
- `--oneline`: Shows each commit on one line.
- `--graph`: Draws an ASCII graph of branch history.
- `--all`: Includes commits reachable from all known refs, including remote-tracking branches.

Example workflow:

```
git switch main
git merge feature-login
git log --oneline --graph --all
```

If the merge succeeds, the changes from `feature-login` are integrated into `main`.

What is a merge conflict?

A **merge conflict** occurs when Git cannot automatically combine competing changes. You must edit the conflicting files, resolve the differences, stage the resolved files, and complete the merge.

For a merge that is still in progress:

```
git status
git add file-name
git merge --continue
```

To cancel an unfinished merge:

```
git merge --abort
```

Only use the commands appropriate to the operation Git reports in `git status`.

---

8. Undoing Changes and Recovering Commits

Git provides different tools for different types of mistakes. Choose carefully because some commands discard changes or rewrite local history.

Restoring files

Command	Description
`git restore file-name`	Discards unstaged changes to a tracked file and restores its contents from the staging area.
`git restore .`	Discards unstaged changes to tracked files in the current directory and its subdirectories.
`git restore --staged file-name`	Removes a file's changes from the staging area without discarding the working-copy changes.

Understanding the command parts

- `restore`: Restores file contents.
- `--staged`: Targets the staging area.
- `.`: Represents the current directory.

Warning: `git restore file-name` can permanently discard uncommitted changes. Check `git diff` first if you are unsure.

Amending a commit

```
git commit --amend -m "Updated message"
```

Updates the most recent commit, replacing it with a revised commit. You can use it to correct the commit message or include additional staged changes.

- `--amend`: Replaces the latest commit with a revised one.
- `-m`: Sets the new commit message.

Avoid amending commits that have already been shared unless you understand the consequences of rewriting history.

Reverting a commit

```
git revert COMMIT_ID
```

Creates a new commit that reverses the changes introduced by the specified commit.

- `revert`: Records a new commit that undoes another commit's changes.
- `COMMIT_ID`: The identifier of the commit you want to reverse.

This is often the safer option for undoing a commit that has already been pushed or shared.

Resetting a commit

Command	Description
`git reset --soft HEAD~1`	Moves the current branch back one commit and keeps the undone commit's changes staged.
`git reset --hard HEAD~1`	Moves the current branch back one commit and resets tracked files and the staging area to that commit's state.

Understanding the command parts

- `reset`: Moves the current branch reference to a different commit.
- `--soft`: Keeps the changes staged.
- `--hard`: Resets the staging area and tracked working files.
- `HEAD`: Refers to the current commit.

- `~1` : Refers to the first parent of the current commit.

Warning: `git reset --hard` can destroy uncommitted work. Use it only when you are certain that you do not need the changes it will overwrite. Avoid rewriting history that others depend on.

Recovering commits with reflog

```
git reflog
```

Shows a local record of recent movements of `HEAD` and branch references. It can help you find commits that seem lost after a reset, amend, or branch switch.

For example, after identifying a commit in the reflog, you can create a recovery branch:

```
git branch recovery COMMIT_ID
git switch recovery
```

Replace `COMMIT_ID` with the actual commit hash from your reflog.

Note: The reflog is local and entries are not guaranteed to be retained forever. Recover important work promptly.

---

9. Stashing Changes

Stashing temporarily saves uncommitted changes so you can switch tasks or branches without making a commit.

Command	Description
`git stash`	Saves eligible tracked-file changes and reverts them from the working tree.
`git stash list`	Lists saved stashes.
`git stash pop`	Applies the latest stash and removes it from the stash list if application succeeds.
`git stash apply`	Applies the latest stash but keeps it in the stash list.

Understanding the command parts

- `stash`: Temporarily stores changes outside the regular commit history.
- `list`: Displays available stashes.
- `pop`: Applies a stash and removes it when successful.
- `apply`: Applies a stash without removing it.

Example workflow:

```
git stash
git switch main
git stash list
git switch feature-login
git stash pop
```

This saves eligible changes, lets you switch branches, and then reapplies the changes.

Important: By default, `git stash` does not include untracked files. To include them, use `git stash -u`. If applying a stash causes conflicts, resolve them like other conflicts. `pop` may leave the stash in the list if application fails.

---

10. GitHub Pull Request Process

A **Pull Request (PR)** is a GitHub feature used to propose that changes from one branch be merged into another. It supports code review, discussion, automated checks, and collaboration.

A typical workflow is:

1. Create a feature branch and make your changes.
2. Stage and commit your changes locally.
3. Push the feature branch to GitHub.
4. Open a Pull Request on GitHub, selecting the source and target branches.
5. Review the proposed changes, discuss feedback, and resolve any requested updates.
6. Run or check the required tests and automated checks.
7. Merge the Pull Request when it is approved and ready.
8. Update your local `main` branch so it reflects the merged changes on GitHub.

A Pull Request is not the same as `git merge`: a merge integrates branch changes, while a Pull Request is a review and collaboration workflow that can lead to a merge.

Quick Reference

Goal	Command
Check the current directory	`pwd`
Check Git version	`git --version`
Check repository status	`git status`
Stage a file	`git add file-name`
Create a commit	`git commit -m "message"`
View commit history	`git log --oneline --graph --all`
Create and switch to a branch	`git switch -c branch-name`
Switch branches	`git switch branch-name`
Merge a branch	`git merge branch-name`
Push to GitHub	`git push -u origin main`
Clone a repository	`git clone URL`
Discard unstaged file changes	`git restore file-name`
Unstage a file	`git restore --staged file-name`
Reverse a commit	`git revert COMMIT_ID`
Inspect local reference history	`git reflog`
Temporarily save changes	`git stash`

Final Tips

- Run `git status` frequently to understand your current state.
- Use meaningful commit messages that describe what changed.
- Review `git diff` before discarding changes.

- Prefer `git revert` when undoing commits that have already been shared.
- Be careful with `git reset --hard`, because it can discard work.
- Use branches to isolate features and fixes.
- Keep your local repository synchronized with the remote repository.
- Practice recovery commands in a test repository before using them on important projects.

Remember: Git is not just a collection of commands. Understanding your working directory, staging area, commits, branches, and remote repository is what allows you to use those commands safely and effectively.

Author:

Borna Etminan

📫 Contact me

- 📞 Phone: +989183820252
- ✉️ Email: borna.etminan34@gmail.com
- 📱 Telegram: https://t.me/BornaEtminan
- ✉️ X acount: https://x.com/BornaEtminan
- ✉️ Kaggle username: https://www.kaggle.com/bornaetminan
- 📘 Blog on Substack: https://substack.com/@bornaetminan